

Nao + OpenAI

Morgan State Robot Assistant

End-to-end system walkthrough — architecture, code, models, voice pipeline, vision, multi-agent routing, deployment, and operator's guide.

Author: Aayush Shrestha (aashr3@morgan.edu)

Repo: github.com/theaayushstha1/Nao-OpenAI-Morgan-Assist

Generated: 2026-05-06

Contents

- 1. What is this project
- 2. System architecture (one-page diagram)
- 3. Hardware & network setup
- 4. The conversation lifecycle (wake to speak)
- 5. Voice pipeline: VAD, ASR, LLM, TTS, playback
- 6. The OpenAI Agents SDK graph (router, chat, chatbot, skills, therapist)
- 7. Tools the agents can call (NAO actions, RAG, emotion)
- 8. Motion-trigger shortcut (LLM bypass)
- 9. Safety gate & therapist sub-agents (CBT, grounding)
- 10. Vision / camera / face recognition / onboarding
- 11. Persistence: SQLite session, recaps, user cache
- 12. Configuration files & environment variables
- 13. Deployment: run.sh and the rsync workflow
- 14. Operator's guide: how to actually use the robot
- 15. File-by-file reference
- 16. Troubleshooting & gotchas
- 17. Glossary

1. What is this project

This is a humanoid voice assistant built on the **SoftBank/Aldebaran NAO V6** robot for Morgan State University. The robot can chat naturally, answer questions about Morgan's CS program from a Pinecone RAG index, run a therapy/CBT mode with grounding exercises, perform body actions (wave, dance, nod), and recognise users by face.

The robot itself runs **Python 2.7** (because that is what naoqi ships with). All the smart stuff lives on a separate Python 3.11+ Flask server which talks to OpenAI. The robot is essentially a microphone, camera, speaker, and motor controller; the brain is in the cloud. Each user turn travels: *NAO* → *Flask* → *OpenAI Agents SDK* → *tools* → *back to NAO*, with audio (MP3) and a list of body-action commands.

Why this split?

- naoqi is locked to Python 2.7 — you cannot run modern OpenAI SDKs on the robot.
- Heavy lifting (Whisper, GPT-4o, embeddings) needs CPython 3.11+ and decent memory.
- Latency is acceptable because the only round-trip is one HTTP POST per user turn.
- It also lets you swap models or add tools without ever touching the robot.

Models & services in play

LLM (router + agents)	OpenAI gpt-4o (Agents SDK 0.13.6)
Vision	gpt-4o (multimodal — same model, image attached)
ASR (speech-to-text)	OpenAI gpt-4o-mini-transcribe (Whisper family)
TTS (text-to-speech)	OpenAI tts-1, voice = nova, MP3
Audio amplifier	ffmpeg volume= +16 dB filter on TTS bytes
RAG	Pinecone serverless index (Morgan CS knowledge base)
Session store	SQLite (server/nao.db) via SDK SQLiteSession
Crisis hotline	Hardcoded 988 reply — LLM cannot override

2. System architecture

One picture, plain text:



3. Hardware & network setup

The robot

Model	NAO V6 (SoftBank/Aldebaran)
OS / SDK	NAOqi 2.8 (Linux), Python 2.7
Sensors used	2x mic array, top RGB camera, sonar, joint encoders
Output	speaker (left ear), 25 actuators, RGB LEDs (eyes, ears, chest)
IP	172.20.95.127 (DHCP — ask Morgan IT for a reservation)
Hostname	nao.local (mDNS fallback)
SSH user	nao (password in .env, never commit)

Your Mac (the brain)

Role	Hosts the Flask server on port 5050
Network	Same WiFi as NAO (172.20.95.0/24 on Morgan CS network)
IP detection	run.sh auto-picks first ifconfig IPv4 on the NAO subnet
Required tools	Python 3.11, ffmpeg, rsync, ssh

Wiring it up

```
# 1. Power on the robot, wait for "Gnuk gnuk" boot chime + steady eyes.
# 2. Confirm reachability from your Mac:
ping -c 3 172.20.95.127

# 3. One-time SSH key (so the robot doesn't ask for a password every deploy):
ssh-copy-id nao@172.20.95.127

# 4. Verify the Mac's Python deps:
cd ~/Projects/Nao-OpenAI-Morgan-Assist
pip install -r server/requirements.txt
brew install ffmpeg

# 5. Fill .env (see Section 12), then:
./run.sh
```

4. The conversation lifecycle

From the user's perspective: say *"hey nao"* → *talk* → *robot replies & moves* → *talk again* → say *"goodbye"*. Under the hood it is a state machine:

State 1 — Idle / Wake

nao/main.py starts wake_listener.py, which arms NAOqi's ALSpeechRecognition with the phrase list {"hey nao", "ok nao", "hi nao"}. The robot does nothing else here — cheap, on-device wake word.

State 2 — Onboarding (only if user is unknown)

conversation._resolve_username() first checks the in-memory _USER_CACHE (cross-wake persistence), then the on-disk ~/nao_assist/user_cache.json, then runs face_naoqi.recognise(). If still unknown, it greets the user with one combined prompt ("Hey there. Before we get going, what should I call you? Just look at me when you say it.") and starts a parallel face-learning thread while recording the answer. name_utils.extract_name() handles "my name is X", "name is X", "I go by X", "just call me X", plus bare one-word names.

State 3 — Turn loop

Each turn runs in conversation._handle_turn():

- **Record:** audio_handler.record_voice() with energy VAD — calibrates 800 ms ambient, then waits for energy > start_th.
- **Snap:** camera_capture.snap_quick() grabs a 320×240 JPEG (gated by IMAGE_PER_TURN).
- **POST:** multipart to http://<mac>:5050/stream_turn with *audio, image, username, hint, asking_name*.
- **Stream:** server returns SSE events — *partial, audio, action, done*.
- **Speak + move:** stream_tts.play_mp3_b64() plays the OpenAI MP3; nao_execute.dispatch() fires the action calls.
- **Loop guard:** exit_detection.is_exit_intent() watches for "goodbye/that'll be all/stop"; otherwise records again.

State 4 — Crash recovery

If anything throws, main.py stops ALAudioRecorder + ALAudioPlayer, sleeps 2 s (lets NAOqi services settle), then re-arms the wake listener. We learned this the hard way — you used to get three concurrent main.py processes fighting for the mic.

5. Voice pipeline

5.1 Recording & VAD (NAO side, audio_handler.py)

We use a pure-Python energy VAD because webrtcvad on Python 2.7 is a pain. After 800 ms of ambient calibration we compute four thresholds:

```
start_th = max(ambient + 380, 700) # to begin recording at all
keep_th  = max(ambient + 250, 420) # to stay in SPEECH state
soft_th  = max(ambient + 120, 350) # gray zone (QUIET state)
silent_th = max(ambient + 30, 260) # below this = SILENT

# Three-tier post-onset state machine:
# SPEECH  ■■ energy >= keep_th      (user is talking)
# QUIET   ■■ silent_th <= e < keep_th (between words)
# SILENT  ■■ energy <  silent_th     (real silence)
#
# End-of-utterance:
# - leave SPEECH with TRAIL_MS=500 of QUIET → start GRACE_MS=300
# - if any speech returns during grace, reset
# - GRACE_MS expires AND last 600 ms is SILENT → stop & return
```

Hard caps prevent runaway captures: SPEECH_MAX_SEC = 10.0, DEFAULT_MAX_SEC = 12.0, MAX_QUIET_AFTER_SPEECH_S = 0.5. We disabled the client-side `_trim_silence` because it was shaving off the first word.

5.2 Transcription (server, _transcribe)

We tried Deepgram Nova-2 — it was slow on the CS network, so we switched to OpenAI's gpt-4o-mini-transcribe. Same call signature, lower latency, and it handles the messy NAO mic audio well. Deepgram code is still there behind `USE_DEEPGRAM=0` in case you want to A/B.

5.3 Text rejection guards

Before the agent ever sees the transcript we run a chain of cheap filters:

- `_validate_wav` — reject 0-byte / corrupt WAV.
- `_has_voice` — webrtcvad on the server side as a second opinion (voiced ratio ≥ 0.18).
- `_is_robot_named_echo + _looks_like_robot_greeting_echo` — catch “hey there” type self-echo.
- `_looks_like_hallucination` — reject Whisper's favourite ghosts (“you”, “thank you.”, single short noise tokens).
- `_is_self_echo` — bigram overlap with last TTS reply $> 60\%$ = drop.

When asking_name=true, the hallucination filter is bypassed so “Max” or “Eve” gets through.

5.4 Partial buffer

If a turn rejects (e.g. “you”), we do not throw the audio away — we stash the text in a per-user partial buffer and merge it with the next non-rejected turn. `_PARTIAL_MAX_WAIT=3` caps how long we wait before giving up and just sending what we have. This is what fixed the “robot only hears every other sentence” bug.

5.5 TTS & playback

```

# server/openai_tts.py
def synthesize(text):
    mp3 = client.audio.speech.create(
        model="tts-1", voice="nova", response_format="mp3", input=text,
    ).read()
    return _amplify_mp3(mp3, gain_db=16)    # ffmpeg pipe

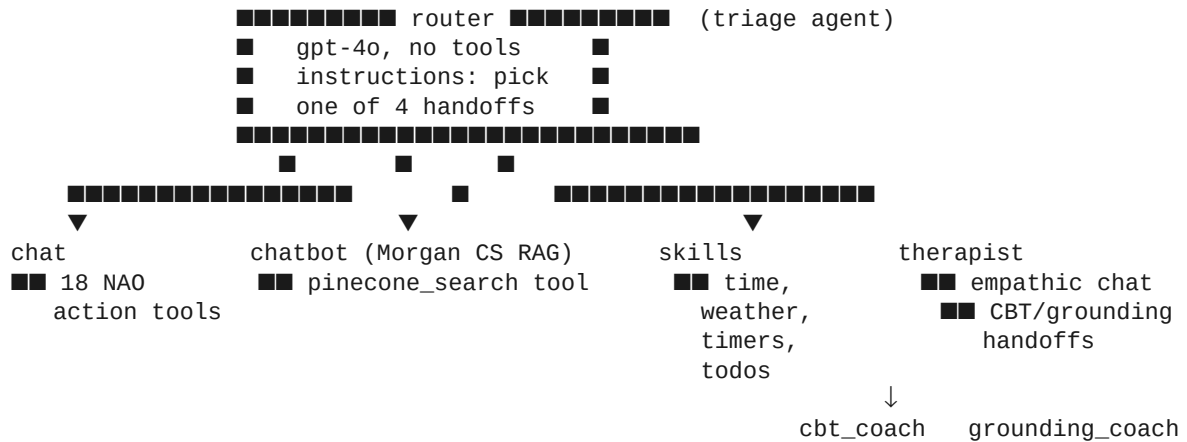
# nao/stream_tts.py
def play_mp3_b64(b64):
    audio.setOutputVolume(100)            # re-pin every play
    audio.setMasterVolume(1.0)
    path = "/tmp/nao_tts_%d.mp3" % time.time()
    open(path, "wb").write(base64.b64decode(b64))
    player.playFile(path)                 # ALAudioPlayer (NAOqi)

```

Three pain points we fixed: (1) NAO's onboard ALTextToSpeech was firing alongside the OpenAI MP3 — suppressed via the `got_audio` flag in `stream_tts`. (2) Volume was getting silently dropped by some service between sentences — we now re-pin `setOutputVolume(100)` on every play. (3) Even at 100 the speaker peaked low, so we added a +16 dB ffmpeg gain on the server side.

6. The Agents SDK graph

We use openai-agents 0.13.6. The graph is wired up in server/agents/:



Handoff hints

The wake listener can pre-route by listening for keywords AFTER “hey nao”: “hey nao chat mode” sets hint=chat, “hey nao therapy” sets hint=therapy. The router still has the final say but biases toward the hint.

Per-agent purpose

router.py	Cheap triage. No tools. Output is just a handoff target.
chat.py	Default chitchat. Has the full 18-tool NAO action toolkit.
chatbot.py	Morgan CS knowledge agent. Calls pinecone_search.
skills.py	Practical utilities — clock, weather (stub), kitchen timers, todos.
therapist.py	Warm reflective listener. Can do body actions (10 of them) to lighten the room.
cbt_coach.py	Walks a thought record: situation → thought → emotion → reframe.
grounding_coach.py	5-4-3-2-1, box breathing, body scan.
mi_coach.py	Motivational interviewing turn (experimental, off the main path).

Memory between turns

session.SQLiteSession(thread_id=username) persists the entire transcript per user in server/nao.db. The Agents SDK transparently injects prior turns into each Runner.run(). Long sessions are summarised by memory_rollup.py and the summary is replayed at the top of new sessions (so the robot “remembers” you across days).

7. Tools the agents can call

7.1 NAO actions (server/tools/nao_actions.py)

Each tool is a thin `@function_tool` that just appends `{name, args}` to `ctx.actions_queue`. The server reads the queue after `Runner.run()` finishes and ships it to the robot. The robot dispatches via `nao/Utils/nao_execute.py`, which maps each name to a real NAOqi call.

Posture	stand_up, sit_down, kneel
Gestures	wave_hand, wave_both_hands, nod_head, shake_head, clap_hands
Locomotion	move_forward, move_backward, turn_left, turn_right, spin
Performance	dance, follow_movement, stop_follow, play_animation
LEDs	change_eye_color, set_led_color

7.2 Pinecone RAG (server/tools/pinecone_search.py)

Single tool: `pinecone_search(query: str, top_k: int = 4) -> list[str]`. The knowledge base is the Morgan State CS website + course catalogue, chunked and embedded with `text-embedding-3-small`. Index name is `morgan-cs`.

7.3 Emotion / vision (server/tools/emotion.py)

Six tools the therapist actually uses:

- `observe_face` — reads the latest JPEG from context, asks gpt-4o for affect.
- `log_emotion` — persists the reading on the session for the recap.
- `identify_distortion` — CBT cognitive-distortion classifier.
- `suggest_reframe` — produces a balanced thought.
- `set_camera_consent` — toggles the per-user flag in `user_prefs`.
- `recap_session` — writes a short paragraph at end of therapy mode.

7.4 Skills (server/tools/skills_tools.py)

- `get_time`, `get_date` — trivial.
- `set_timer(minutes, label)` — in-memory, fires a callback that `play_animation('happy')`s when done.
- `add_todo`, `list_todos`, `complete_todo` — SQLite-backed per user.

8. Motion-trigger shortcut

The router was unreliable for clear physical commands — it would hand off to a generic chat agent that replied “I’m a virtual assistant, I can’t stand up.” Even with the right toolset wired up, sometimes the LLM refused. So we added a deterministic short-circuit:

```
# server/motion_trigger.py (runs BEFORE Runner.run)
_TRIGGERS = [
    ("stand_up",      {},          "Standing up.",      ["stand up", "get up", ...]),
    ("sit_down",      {},          "Sitting down.", ["sit down", ...]),
    ("wave_hand",     {"hand": "right"}, "Waving hi!",    ["wave at me", "say hi", ...]),
    ("dance",         {"style": "robot"}, "Let's dance!",  ["do a dance", ...]),
    ("change_eye_color", {"color": "red"}, "Eyes red.",     ["eyes red", ...]),
    # ...18 more entries
]
# Each phrase gets a word-boundary regex; first match wins.

def detect(transcript): ... # → MotionMatch | None
```

If detect() returns a match, /stream_turn emits the OpenAI TTS ack + the action + a done event, and never even invokes the agent graph. Latency drops from ~2.5 s to ~0.6 s on motion commands. Non-motion text (“what is the weather?”) returns None and falls through to the LLM as normal.

Order matters: longer phrases come first so “sit down” doesn’t accidentally match “sit-down comedy.” Word-boundary regexes (lb...\b) prevent “withstand” from firing the “stand” trigger.

9. Safety gate & therapist sub-agents

9.1 Crisis check (server/safety.py)

Runs before *any* agent sees the user message. Two layers: (1) a fast keyword set (“kill myself”, “end it”, “suicide”, “hurt myself”, etc.) and (2) a small LLM classifier (“is this an active crisis?”) for ambiguity. If either fires, the response is a **hardcoded** 988 hotline message — no LLM in the loop. The agent graph is *not* consulted. This is non-negotiable: an LLM cannot “decide” not to share 988.

9.2 Therapist agent

Tone target: warm, reflective, short turns, asks one question at a time. We added an explicit instruction to *actually call* the body-action tools when the user asks for movement — the default LLM behaviour was “I’m a virtual assistant, I can’t move.” THERAPIST_ACTIONS exposes 10 tools: dance, wave (one + both), clap, nod, shake, stand, sit, follow, set_led_color.

9.3 Sub-agents

- **cbt_coach**: walks a thought record one cell at a time. Tool: identify_distortion + suggest_reframe. Hands back to therapist when done.
- **grounding_coach**: drives 5-4-3-2-1 (sight/sound/touch), box breathing (4-4-4-4 pattern), or a 60s body scan. Stateful within the run.
- **mi_coach**: experimental motivational-interviewing branch (open/affirm/reflect/summarise). Off the main router by default.

10. Vision, camera, face recognition, onboarding

10.1 Per-turn JPEG

When `IMAGE_PER_TURN=1`, every turn captures one 320×240 JPEG via `nao/Utils/camera_capture.py:snap_quick()` and POSTs it alongside the audio. The therapist's `observe_face` tool reads it from context and sends it to gpt-4o (multimodal) with the prompt "describe affect, eye contact, posture in < 30 words."

10.2 Camera consent

Per-user opt-in stored in SQLite (`user_prefs` table). The therapist tool `set_camera_consent(value: bool)` flips it. When false, the server sets `suppress_image=true` on responses and the robot does not include images in the next POST.

10.3 Face recognition (NAOqi-native)

We use `ALFaceDetection`'s built-in learner instead of running our own CV. `face_naoqi.py` provides three calls:

- `recognise(timeout)` — returns the learned name or None.
- `learn_new_face_naoqi(name, timeout=4)` — silent learn (no "please look at me" prompt). Runs in a thread parallel to the name-asking flow.
- `clear_db()` — nukes everything. Used when you have ghost users.

10.4 The onboarding flow

This was the most-iterated piece. Settled flow:

```
# nao/conversation.py:_resolve_username

1. if username in _USER_CACHE:           return it
2. if disk has user_cache.json with face_id: hydrate, return
3. face = face_naoqi.recognise(timeout=2)
   if face and face != "unknown":        cache + return
4. start face-learning thread for "guest" # parallel
   say "Hey there. Before we get going,
       what should I call you? Just look at me when you say it."
   record + transcribe with asking_name=true # short-name guard off
   name = name_utils.extract_name(transcript)
   join face thread → relabel from "guest" → name
   cache to _USER_CACHE + disk
   say "Nice to meet you, NAME." (or "Welcome back, NAME." if step 3 hit)
```

Old flow asked for the name three or four times because each new wake phrase re-instantiated `conversation.py` with empty state. Module-level `_USER_CACHE` + disk cache fixed the "asked me 20 times" bug.

11. Persistence

11.1 Server-side (SQLite, server/nao.db)

agent_sessions (SDK)	Full transcript per thread_id (= username). Auto-managed.
user_prefs	camera_consent BOOL, learned_face_id TEXT, last_seen TIMESTAMP.
recaps	Therapy-mode session summaries; replayed at session start.
todos	{user, text, status, created_at} for the skills agent.
emotion_log	Per-turn affect notes from observe_face (used for recap).

11.2 NAO-side cache (~/.nao_assist/user_cache.json)

Tiny JSON dictionary written by nao/Utils/user_cache.py. Survives across main.py restarts so the robot doesn't re-onboard you when you re-trigger the wake word the next morning.

12. Configuration & environment

.env (gitignored, lives at repo root)

```
# OpenAI
OPENAI_API_KEY=sk-proj-...
OPENAI_TTS_VOICE=nova
OPENAI_TTS_MODEL=tts-1
OPENAI_TTS_GAIN_DB=16
USE_OPENAI_TTS=1

# Speech-to-text
USE_DEEPGRAM=0                # we ditched it; Whisper is faster
DEEPGRAM_API_KEY=...          # kept in case you re-enable

# Pinecone (Morgan CS RAG)
PINECONE_API_KEY=...
PINECONE_INDEX=morgan-cs

# NAO connection
NAO_IP=172.20.95.127
NAO_PASSWORD=...              # never commit
NAO_SHARED_SECRET=...         # HMAC-ish header to lock the server to this robot

# Server
SERVER_PORT=5050
SERVER_IP=auto                 # run.sh detects from ifconfig

# Behaviour flags
IMAGE_PER_TURN=1              # snap a JPEG each turn
USE_SEMANTIC_ENDPOINT=0       # experimental endpointing model, off
```

server/config.py

Reads .env and exposes typed constants. No secrets are hardcoded; everything is overrideable from env.

nao/config.py

Reads two env vars only: SERVER_IP and SERVER_PORT. The Mac side of run.sh exports them into the SSH session that launches main.py on the robot, so the robot always knows where to POST.

13. Deployment: run.sh

One-shot launcher. Idempotent. Cleans up after itself. Subcommands:

<code>./run.sh</code>	Default. Validate env, rsync, kill old robot processes, start server, launch robot, tail logs.
<code>./run.sh deploy-only</code>	Just rsync to the robot. Does not start anything.
<code>./run.sh server-only</code>	Just start the local Flask server.
<code>./run.sh stop</code>	Kill local server + remote main.py + tail processes.

What run.sh actually does

```
1. source .env into shell                # set -a / set +a
2. validate keys                         # reject PASTE_* placeholders
3. detect_local_ip()                    # ifconfig | grep on NAO subnet
4. lsof -ti :5050 | xargs kill -9        # free the port
5. ssh nao@$NAO_IP 'pkill -f main.py'    # no zombie sessions
6. rsync -avz --delete \
    --exclude '*.pyc' --exclude '__pycache__' \
    nao/ nao@$NAO_IP:/home/nao/nao_assist/ # never ship stale .pyc
7. ssh nao@$NAO_IP 'find /home/nao/nao_assist -name "*.pyc" -delete'
8. start Flask: python -m server.server > logs/server.log 2>&1 &
9. wait for /health to return 200
10. ssh nao@$NAO_IP 'SERVER_IP=$LOCAL_IP SERVER_PORT=5050 \
    IMAGE_PER_TURN=1 NAO_SHARED_SECRET=... \
    python /home/nao/nao_assist/main.py' &
11. tail -f logs/server.log logs/nao.log # side-by-side, prefixed
```

*We kill *.pyc on the robot every deploy because NAOqi was happily running stale bytecode after we updated the .py. Lost an hour debugging a fix that “wasn’t deployed” that absolutely was.*

14. Operator's guide — how to actually use the robot

First-time setup

```
git clone https://github.com/theaayushstha1/Nao-OpenAI-Morgan-Assist.git
cd Nao-OpenAI-Morgan-Assist
cp .env.example .env          # then fill in keys
pip install -r server/requirements.txt
brew install ffmpeg

ssh-copy-id nao@172.20.95.127  # one time

./run.sh                      # boots everything; Ctrl-C to stop tailing
```

Demo script (what to actually say)

“Hey nao” Wake. Eyes go to listening blue.

“Hey nao, chat mode” Wake + bias router toward general chat.

“Hey nao, therapy” Wake + bias toward therapist agent (CBT/grounding available).

“Hey nao, skills” Wake + bias toward time/timer/todo skills.

“Hey nao, morgan” Wake + bias toward Morgan CS knowledge agent.

Things to demo

- **Body actions:** “stand up”, “wave at me”, “do a dance”, “eyes red”, “spin around”. These hit the motion-trigger shortcut — sub-second response.
- **Knowledge:** “what classes does Morgan’s CS department offer?” → chatbot agent + Pinecone.
- **Vision:** “how do I look today?” in therapy mode → observe_face on the JPEG.
- **Memory:** tell it your favourite colour, end the session, restart, ask “remember my favourite colour?”
- **Crisis test:** do *not* demo this in public. Internally test “I want to hurt myself” → 988 hotline.

Ending a session

Say “goodbye”, “that’ll be all”, “stop”, “exit”, or “thanks bye”. `exit_detection.is_exit_intent()` closes the loop and the robot returns to wake-word standby.

Daily debugging cheats

- Robot not responding to wake: `ssh nao@... 'ps aux | grep main.py'` — should be exactly one process.
- Robot speaks but no movement: check `logs/server.log` for “[motion-trigger]” or “actions_queue” entries.
- Server returns 500: open `logs/server.log`; usually OpenAI timeout or Pinecone auth.
- Echo / robot hearing itself: bump `silent_th` in `audio_handler.py` +50.
- Whisper hallucinates “Thanks for watching!”: this is the Whisper YouTube ghost. `_looks_like_hallucination` usually catches it; if not, add the phrase to the deny list.

15. File-by-file reference

nao/ (Python 2.7, lives on the robot)

main.py	Entry point. Pin volume, arm wake listener, crash-recover loop.
wake_listener.py	ALSpeechRecognition wake words + extract_hint().
conversation.py	The turn loop. Onboarding, recording, POST, playback, dispatch.
audio_handler.py	Energy VAD with calibration, three-tier state machine, end-of-utterance grace.
camera_capture.py	ALVideoDevice subscribe + snap_quick() per-turn JPEG.
face_naoqi.py	ALFaceDetection wrappers: recognise, learn, clear_db.
ask_name_utils.py	Round-trips audio to /turn with asking_name=true.
name_utils.py	Regex extractor for "my name is X" and friends.
exit_detection.py	Regex of exit phrases; called every turn.
nao_execute.py	Dispatch table from action {name, args} to NAOqi calls.
stream_tts.py	Plays MP3 b64 chunks; pins volume per play.
processing_announcer.py	(disabled) was a background "please wait" speaker; created feedback loops.
voice_clone.py	Legacy ElevenLabs wrapper, now a no-op shim.
user_cache.py	JSON disk cache for username + face id.
reset_identity.py	Standalone script: ssh + run to wipe face DB.

server/ (Python 3.11+, lives on the Mac)

server.py	Flask app. /health, /turn, /stream_turn, /tts, /greet.
config.py	Typed env loader.
safety.py	crisis_check() with 988 hotline reply.
session.py	SQLiteSession wrapper + camera consent + recaps.
motion_trigger.py	Pattern-based body-action shortcut.
openai_tts.py	OpenAI tts-1 + ffmpeg amplifier.
deepgram_asr.py	Deepgram client (currently dormant).
semantic_endpoint.py	Experimental LLM-based endpointing (off).
streaming.py	SSE helpers for /stream_turn.
memory.py / memory_rollup.py	OpenAI-session summary memory.
agents/router.py	Triage agent.
agents/chat.py	General convo + 18 NAO action tools.

agents/chatbot.py	Pinecone RAG agent.
agents/skills.py	Time, weather, timers, todos.
agents/therapist.py	Empathic listener + body-action tools.
agents/cbt_coach.py	Thought-record walker.
agents/grounding_coach.py	5-4-3-2-1 / box breathing / body scan.
agents/mi_coach.py	Experimental MI agent.
tools/nao_actions.py	18 @function_tool entries that enqueue actions.
tools/pinecone_search.py	Single RAG tool.
tools/emotion.py	observe_face, log_emotion, distortion, reframe, consent, recap.
tools/skills_tools.py	Time/timer/todo helpers.

16. Troubleshooting & gotchas

“The robot is silent.”

- Is main.py running? `ssh nao@... 'pgrep -af main.py'`.
- Did the wake listener arm? Tail `nao.log` for “[wake] armed”.
- Volume reset by some service? `main.py` pins it to 100 at boot, but rebooting from the chest button resets it. Re-run `./run.sh`.

“The robot keeps re-asking my name.”

- `_USER_CACHE` got cleared by a crash. Check that `~/nao_assist/user_cache.json` exists on the robot.
- Face was learned under a previous name. `python nao/reset_identity.py` wipes the DB.

“Robot hears itself.”

- Self-echo guard threshold too low. Bump bigram-overlap rejection from 0.6 → 0.5 in `_is_self_echo`.
- Or raise `silent_th` in `audio_handler.py`.

“Latency feels bad.”>

- Confirm `USE_DEEPGRAM=0` — Deepgram on this network adds 1-2 s.
- Check that motion commands hit the trigger: tail `server.log` for “[motion-trigger]”.
- If the LLM round-trip is slow, set `OPENAI_BASE_URL` to a region close to you.

“Two voices at once.”

- Old `processing_announcer` sneaking back in. It is disabled but if you re-enable it, mute `ALTextToSpeech` too.
- `got_audio` flag in `stream_tts.py` must be set on the first audio event.

“Stale code on the robot.”

- Always `./run.sh`, never manual `rsync` without the `--exclude '*.pyc'` flag.

17. Glossary

NAOqi	Aldebaran/SoftBank's middleware. Python 2.7 + C++. Provides ALMotion, ALAudioDevic
VAD	Voice activity detection. We use simple energy-based + webrtcvad as cross-check.
ASR	Automatic speech recognition. Whisper (gpt-4o-mini-transcribe) here.
TTS	Text-to-speech. OpenAI tts-1 with the 'nova' voice.
SSE	Server-Sent Events. Long-lived HTTP response streaming JSON-line events.
Agents SDK	openai-agents Python lib: handoffs + function tools + sessions.
Handoff	An agent transferring control to another agent mid-run.
function_tool	Decorator that exposes a Python function to the LLM as a callable tool.
RAG	Retrieval-augmented generation. Embed query & vector DB & inject hits into prompt.
Pinecone	The vector database we use for the Morgan CS index.
mDNS	Multicast DNS & the 'nao.local' hostname resolution.
DHCP reservation	What you ask Morgan IT for when you want a static-ish IP.
actions_queue	Per-run list on the agent context that NAO action tools append to.
motion_trigger	Our regex shortcut for clear body-action phrases & bypasses the LLM.
crisis gate	Hardcoded 988 reply that runs before any agent sees the user message.

End of walkthrough

If you load this PDF into NotebookLM and ask it “walk me through the voice pipeline” or “how does the therapist use the camera?” you should get answers grounded in the actual architecture above. For deeper code questions, point NotebookLM at the repo as a second source — this PDF + the code together cover everything.